Polymarket Market-Making — Backtesting Infrastructure

A build-and-split proposal · Justin & Alvaro · June 2026

In one line: *we build a single engine that runs the* ***same code in backtest and live****, so 'reliable' means precisely that the two agree. The bot is* ***strategy-agnostic*** *— the machine is built now with a placeholder quoter and a real strategy plugs in later, so* ***no finished strategy is needed to start.***

SITUATION

We want to know whether a passive market-making strategy on Polymarket is viable. The real blocker is not strategy ideas, it is a **trustworthy backtest**. For a passive maker, the two quantities that decide profit — our **queue position** (how much size rests ahead of our order) and our **fill rate** — cannot be measured from public data; we only learn them by quoting live. A backtest of a passive maker is therefore only as trustworthy as its agreement with real fills, which is why backtesting and live validation have to be built as a single loop.

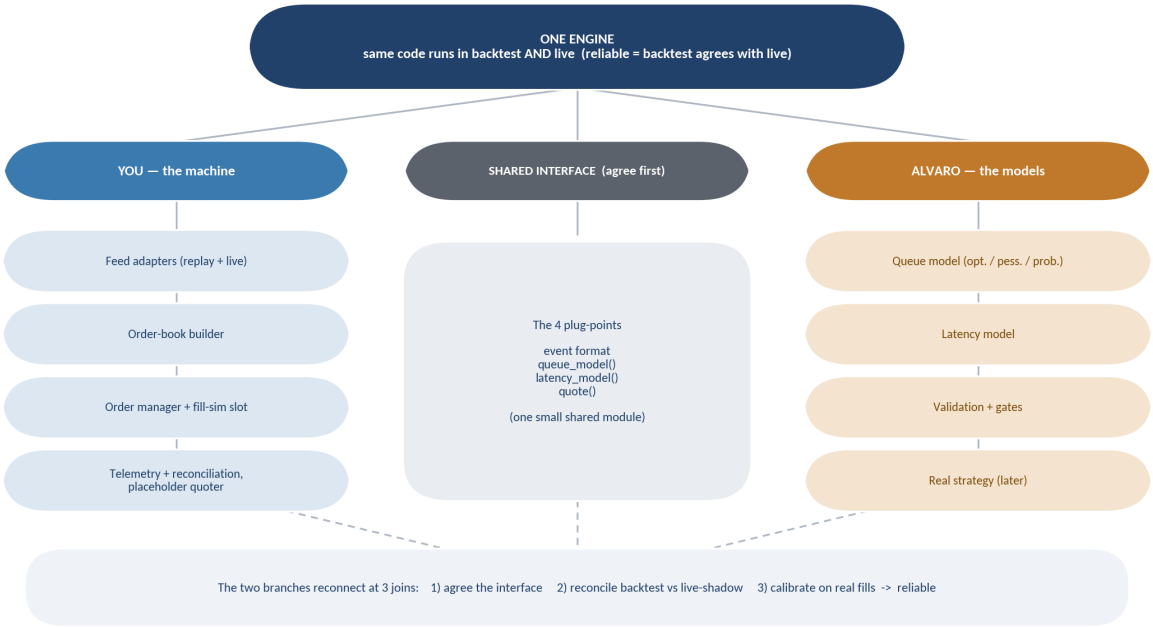
TASK

Build **one strategy-agnostic engine that runs identical code in backtest and live**, then divide the build cleanly so we can work in parallel. Because the strategy is just one pluggable function, **we do not need it yet**: a placeholder symmetric quoter is enough to build and stress the whole machine.

Scope for this phase: *deliver the machine and prove it reliable on slow politics markets. A real strategy, any profitability claim, and the faster markets (crypto and in-play sports, where latency also bites) come after the machine is trusted.*

ACTION

We split the system into a **machine** (Justin) and a set of **models** (Alvaro) that plug into it at a small shared interface. The tree shows the separation: everything in one column is owned by one person, the two branches meet only at the interface, and they reconnect at the three joins.



Two independently-owned branches; one shared interface; three points where the work rejoins.

Who owns what

YOU — the machine	ALVARO — the models
• Feed adapters (replay + live) • Order-book builder • Order manager (place / cancel / replace) • Fill-simulator slot + live / shadow execution • Telemetry + reconciliation harness • Placeholder quoter	• Queue model (optimistic / pessimistic / probabilistic, with a calibration hook) • Latency model • Validation + overfitting gates • The real quoting strategy (later)

The VPS is just a shared cloud box collecting live L2; both of us have access. It is the data source, nothing more.

The shared interface — agree this first

- **Event format** — the shape of one captured data event (book update, price change, trade, our order, fill); essentially what the capture already writes, fixed so replay and live are identical.
- **queue_model(book_state, our_order) → fill?** — Alvaro's model, called by the fill simulator.
- **latency_model() → round_trip**
- **quote(book_state, inventory) → orders** — the strategy slot; placeholder now, real later.

Plus one agreement: **which numbers must match** between backtest and live, and how close — fill rate, position path, and equity, over the same dates, within an agreed tolerance. That definition is what makes 'reliable' testable.

The three joins

Join	You bring	Alvaro brings	Done when
1 Agree the interface	Half a day, together.	Half a day, together.	The four plug-points exist as stub signatures we both build against, plus the agreed list of reconciliation numbers.
2 Reconcile backtest vs live-shadow	Engine runs end to end in replay and live-shadow (observing, no real orders); placeholder quoter in both; comparable logs.	Queue and latency models implemented behind the interface (runnable, not just the maths).	The same code gives identical decisions in backtest and live-shadow, and the rebuilt book matches the live feed. Proves consistency and same-code-path, not the fill model yet.
3 Calibrate on real fills	Live path places real 1-contract quotes on one market; logs queue rank, fills, post-fill drift, round-trip latency.	Fit the queue model and latency constant to those fills; re-run the calibrated backtest.	Backtest fill estimates collapse toward the live-measured rate. **The backtest is now reliable.**

WHY IT MAKES SENSE

- **Strategy-agnostic, so we do not need a strategy yet.** The strategy is one pluggable function; the placeholder quoter is also the standard benchmark used to A/B-test a real strategy later, and it is enough to prove the machine works.
- **Same code path.** Running identical code in backtest and live is the only way the reconciliation is meaningful, and the only way a backtest that passes actually transfers to production.
- **Clean parallelism.** The machine and the models live in different files and meet only at the interface, so we each own our half; a branch per person keeps us out of each other's working copies.
- **It matches who does what.** Justin owns the executable machine and live infrastructure; Alvaro owns the modelling and statistics.

NEXT STEPS (THIS WEEK)

- **Both:** agree the four plug-points and the reconciliation numbers.
- **Justin:** scaffold the engine and placeholder quoter; get one market replaying through it, and the same code connecting to the live VPS stream in shadow — that smoke proves the same-code-path holds.
- **Alvaro:** draft the queue and latency models against the stub interface, prototyping on the week of L2 we already have.

CAVEATS

- Until the reconciliation passes, a backtest number is a **breakeven fill rate**, never a profit claim.
- The week of L2 we have is enough to **build and test the machine**, not to declare an edge; the VPS now collects continuously going forward.
- The real unknown is whether the **queue model calibrates cleanly against live fills** at Join 3 — that is the moment we learn whether the approach holds; everything before it is conditional on a queue assumption.